

NumPy (Numeric Python)

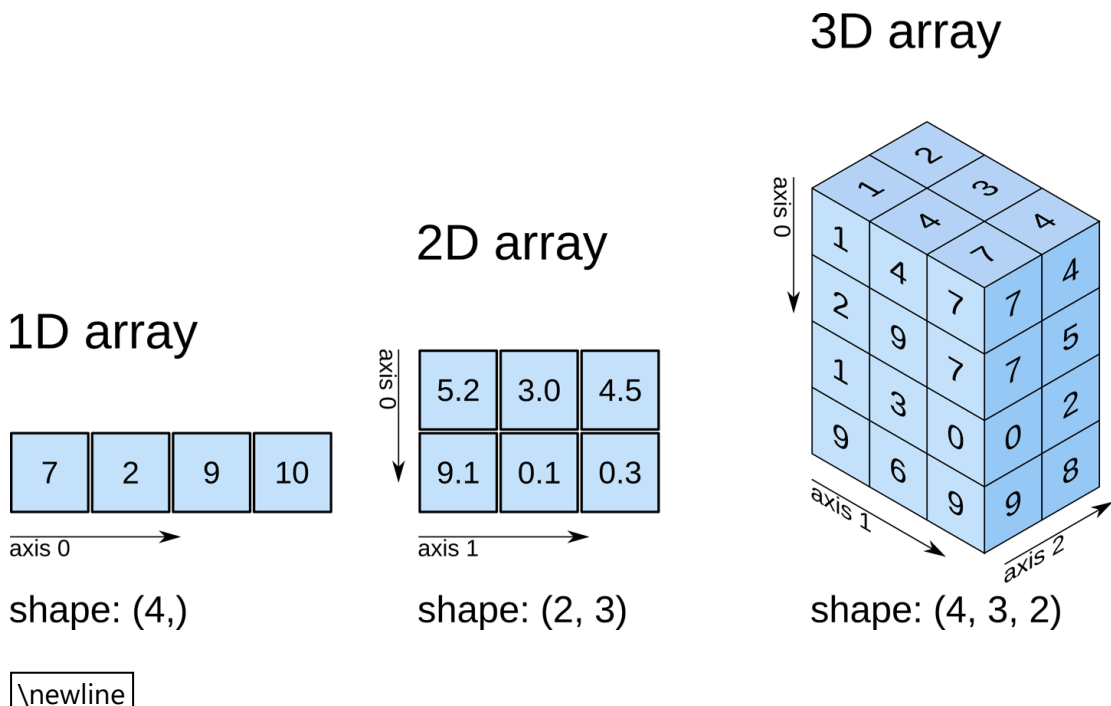
It is a [package](#) which provides us with speed in mathematical calculation on our data.

Provides the **NumPy Array** as an alternative to the regular **Python List** to execute element-wise (i.e. element by element) operations among the elements of a sequence of numbers as well as execute arithmetic operators, such as $+$, $-$, $*$ and $/$; and relational operations, such as $>$, $<$, $=$ among vectors, matrices and tensors.

NumPy Array

In computer science an **Array** is a data structure (class) that stores values very much like lists, except that the type of objects stored in them is constrained, that is, all of the elements should be of the same type. The biggest advantage of using an array is that it can be used to store vectors, matrices and tensors; and therefore, we can execute element-wise operations and vector/matrix operations using arrays.

The NumPy package provides a container for the array data structure, this is called the [NumPy Array](#). Arrays may have several dimensions; for example, a 1-dimensional array (1D array) can be used to store a vector with the coordinates of a point in 3D space. In NumPy dimensions are also called axes. A 2-dimensional array has two corresponding axes: the first running vertically downwards across rows (axis 0), and the second running horizontally across columns (axis 1).



The computational representation of the vector is, `array([7, 2, 9, 10])`, it has just one axis. In addition, this axis has 4 elements in it, so we say it has a length of 4. In the next example, we want to store a matrix (2 dimensions) in an array, so the array has 2 axes.

The first axis (axis 0) has a length of 2, the second axis (axis 1) has a length of 3. This array has 6 components.

*Matrix : *MathematicalRepresentation*

$$A_{m,n} = \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \end{bmatrix}$$

Example

$$\begin{bmatrix} 5.2 & 3.0 & 4.5 \\ 9.1 & 0.1 & 0.3 \end{bmatrix}$$

**ComputationalRepresentation*

`array([[5.2, 3.0, 4.5], [9.1, 0.1, 0.3]])`

Tensor - nD array

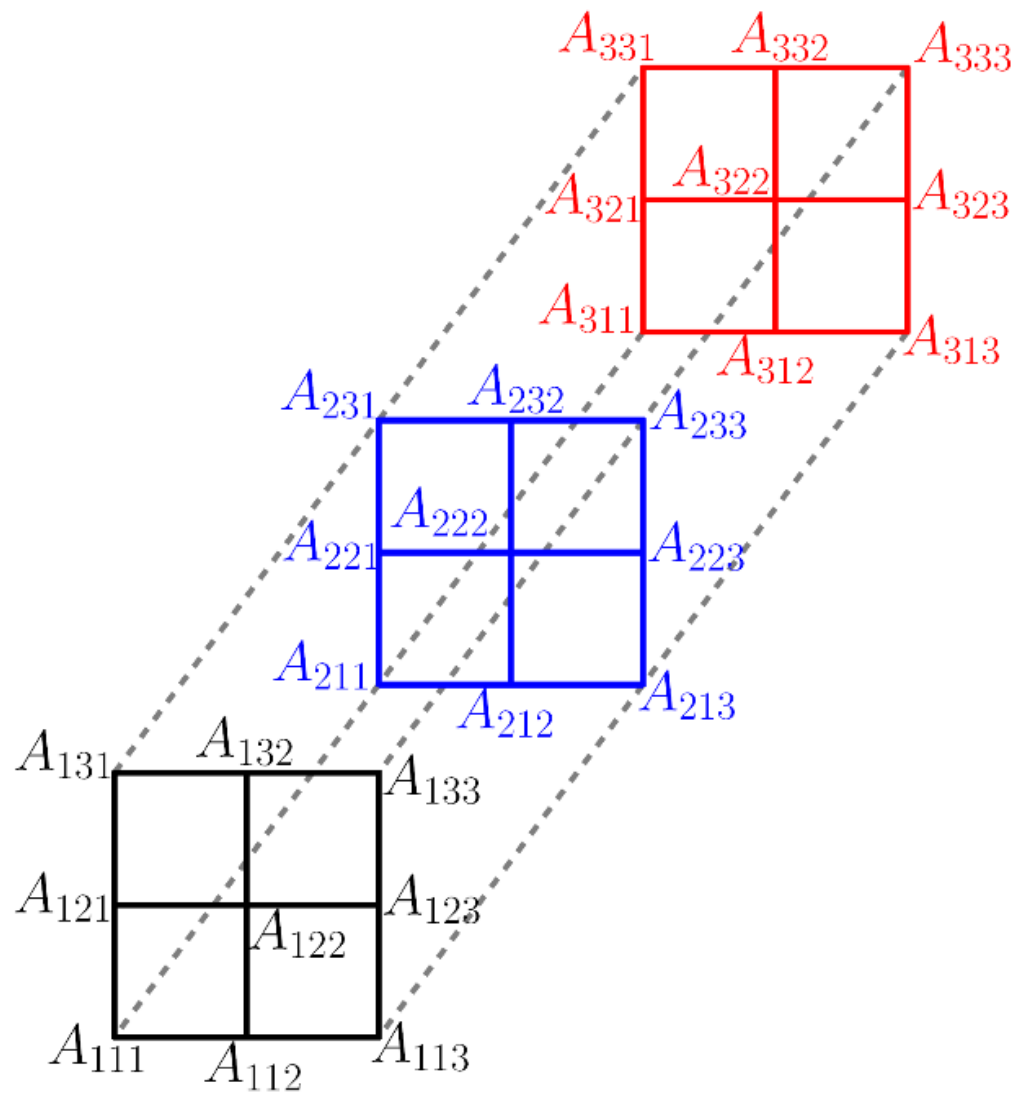
A tensor is an array of mathematical objects (usually numbers or functions) which transforms according to certain rules under coordinates change. In an nD space (ex. 2D and 3D), a tensor of rank- k has n^k components which may be specified with reference to a given coordinate system. Accordingly, a scalar, such as temperature, is a rank-0 tensor with (assuming a 3D space) $3^0 = 1$ component, a vector, such as force, is a rank-1 tensor with $3^1 = 3$ components, and stress is a rank-2 tensor with $3^2 = 9$ components. So the rank is independent of the number of space dimensions (nD).

Notes: Stress is a physical quantity that expresses the internal forces that neighbouring particles of a continuous material exert on each other

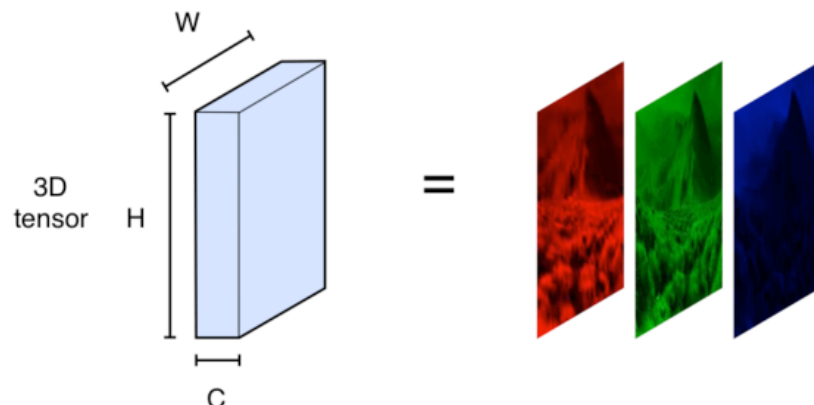
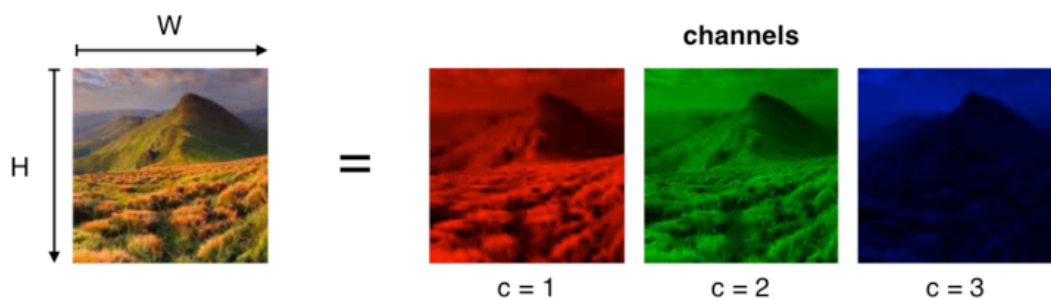
A very common structure is the rank-2 tensor, a 3x3 matrix which represents the map of one 3D thing into another 3D thing. "General Relativity" uses a lot of rank-2 tensors in 4 dimensions of space time, represented by a 4x4 matrix.

In the next figure is graphically illustrated the structure of a rank-3 tensor in a 3D space. (Taken from Taha Sochi, Principles of tensor calculus).

QUESTION: What are the axes' lengths of this tensor?



Rta: Este tensor tiene 3 ejes donde cada eje tiene una longitud de 3



Importing a NumPy Array

In order to use Numpy Array we need first to import the Numpy Package. We import it as an object called "np". Usually this object is called "np" but it can get any name.

```
In [3]: # Usually, we first create a "np" object of the numpy package.
import numpy as np
```

Creating a NumPy Array

In order to store a vector (1-dimensional array), a matrix (2 dimensional array) or a tensor (n dimensional array); we need to create an object of the NumPy's array class called ndarray. <class 'numpy.ndarray'> (N-dimensional array). It is also known as "array".

```
In [4]: #Let's create an array object that stores a vector
#vector_object = np.array(1,2,3,4)    # WRONG
#In order to create an array you need to provide lists
new_array = np.array([1,2,3,4]) # RIGHT
print(type(new_array))
new_array
```

```
<class 'numpy.ndarray'>
```

```
Out[4]: array([1, 2, 3, 4])
```

How to encapsulate (or store) either a vector or a matrix into a Array

```
In [5]: vector_object = np.array([1,2,3,4])
print(type(vector_object))
vector_object
```

```
<class 'numpy.ndarray'>
```

```
Out[5]: array([1, 2, 3, 4])
```

```
In [6]: matrix_object = np.array([[1,0,0],[0,1,2]])
matrix_object
```

```
Out[6]: array([[1, 0, 0],
               [0, 1, 2]])
```

```
In [7]: #We want to check the dimensions of the previous vector and matrix
print(vector_object.ndim)
print(matrix_object.ndim)
```

```
1
2
```

It is expected that a vector has a dimension equals to 1, and a matrix a dimension equals to 2

```
In [8]: #Now Let's figure out the length of the axes
print(vector_object.shape)
```

```
print(matrix_object.shape)
```

```
(4,)
(2, 3)
```

Matrix mathematical concepts

QUESTION: What's the difference between element-wise multiplication and matrix multiplication?

$$\begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{bmatrix} \begin{bmatrix} b_1 & b_2 & b_3 \\ b_4 & b_5 & b_6 \\ b_7 & b_8 & b_9 \end{bmatrix} = \begin{bmatrix} c_1 & c_2 & c_3 \\ c_4 & c_5 & c_6 \\ c_7 & c_8 & c_9 \end{bmatrix}$$

Element-wise multiplication

$$c_1 = a_1 b_1$$

$$c_2 = a_2 b_2$$

Matrix multiplication

$$c_1 = a_1 b_1 + a_2 b_4 + a_3 b_7$$

$$c_2 = a_1 b_2 + a_2 b_5 + a_3 b_8$$

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \dots + a_{in}b_{nj} = \sum_{k=1}^n a_{ik}b_{kj}$$

QUESTION: What's the difference between element-wise multiplication and matrix multiplication?

Rta: La multiplicación elemento a elemento multiplica valores correspondientes y no combina información. La multiplicación matricial calcula cada elemento como una suma de productos fila-columna. Conceptualmente, la primera escala datos; la segunda representa transformaciones lineales.

Python Lists can't execute element-wise operations

To illustrate it, let's try to calculate the body mass index (BMI) for several people

$$\text{BMI} = \frac{\text{weight}_{\text{kg}}}{\text{height}_{\text{m}}^2}$$

```
In [9]: import numpy as np
```

```
# Datos originales
height = [1.73, 1.56, 1.67, 1.69, 1.66]
weight = [80.3, 55.6, 65.5, 70.6, 62]

# 1. Convertimos a NumPy arrays para corregir el TypeError
height_array = np.array(height)
weight_array = np.array(weight)

# 2. Calculamos el BMI (esto ahora funcionará correctamente)
bmi = weight_array / height_array ** 2

# 3. EXERCISE: Creamos el array booleano "light" para BMI < 23
light = bmi < 23

# Mostramos el resultado
print("BMI:", bmi)
print("Light:", light)
```

```
BMI: [26.83016472 22.84681131 23.48596221 24.71902244 22.4996371 ]
Light: [False  True False False  True]
```

The output returns `TypeError: unsupported operand type(s) for ** or pow(): 'list' and 'int'`

This means we are not able to do this calculation that way. Probably we will need to do something like this:

```
In [10]: %%time
height = [1.73, 1.56, 1.67, 1.69]
# mass or weight
weight = [80.3, 55.6, 65.5, 70.6]

bmi_list = []
for i in range(4):
    bmi_list.append(weight[i] / (height[i]**2))

print(bmi_list)
```

```
[26.830164723178186, 22.846811308349768, 23.485962207321883, 24.719022443191765]
CPU times: total: 0 ns
Wall time: 1.52 ms
```

Array Concepts

Vectorization

Vectorization efficiently applies operations to groups of elements. Using vectorization we can execute element-wise operations

Unlike lists, numpy arrays can execute element-wise operations, such as arithmetic (+, -, *, /) and relational (>, <, ==, !=) operations among arrays and even with numerical values.

We need first to convert the lists to arrays.

```
In [11]: %%time
height = [1.73, 1.56, 1.67, 1.69, 1.66]
# mass or weight
weight = [80.3, 55.6, 65.5, 70.6, 62]
#Create the arrays
height_array = np.array(height)
weight_array = np.array(weight)

#Aritmethic operarion between an array and a numerical value
square_height = height_array ** 2

#Arithmetic operation among arrays
bmi = weight_array / height_array ** 2
bmi
#print(bmi)
```

CPU times: total: 0 ns

Wall time: 0 ns

```
Out[11]: array([26.83016472, 22.84681131, 23.48596221, 24.71902244, 22.4996371 ])
```

Indexing

Indexing an array returns single elements, subarrays or elements that satisfy a specific condition.

Indexing for copying, with mask

EXCERCISE: Create a boolean numpy array where each element of the array should be True if the corresponding person's BMI is below 23. You can use the `<` operator for this. Name the new array as "light".

```
In [12]: import numpy as np

# Datos originales
height = [1.73, 1.56, 1.67, 1.69, 1.66]
weight = [80.3, 55.6, 65.5, 70.6, 62]

# 1. Convertimos a NumPy arrays para corregir el TypeError
height_array = np.array(height)
weight_array = np.array(weight)

# 2. Calculamos el BMI (esto ahora funcionará correctamente)
bmi = weight_array / height_array ** 2

# 3. EXCERCISE: Creamos el array booleano "light" para BMI < 23
light = bmi < 23

# Mostramos el resultado
print("BMI:", bmi)
print("Light:", light)
```

BMI: [26.83016472 22.84681131 23.48596221 24.71902244 22.4996371]

Light: [False True False False True]

```
In [13]: print(light)
```

```
[False True False False True]
```

Indexing for viewing, with mask

Now, let's print out a numpy array with the BMIs of only the people whose BMI is below 23. We use "light" inside square brackets to do a selection on the "bmi" array. We can use square brackets to subset numpy arrays.

```
In [14]: print(bmi[light])
```

```
[22.84681131 22.4996371 ]
```

Subsetting Numpy Arrays (Indexing)

Let's define a matrix A , where we want to get the A_{23} value

$$A = A_{m,n} = \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \end{bmatrix}$$

$$A = A_{m,n} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 2 \end{bmatrix}$$

Remember that in Numpy Arrays the index of the first element is 0, and the last index is $N - 1$, where N is the length of the axes.

```
In [15]: A = np.array([[1,0,0],[0,1,2]])
print(A)
print(A[1][2])
#As alternative we can also get that value with this syntax
print(A[1,2])
```

```
[[1 0 0]
 [0 1 2]]
2
2
```

Slicing and dicing

Subsetting several elements at once

```
subset = my_array[row, start_column:end_column]
```

```
subset = my_array[start_row:end_row, column]
```

```
subset = my_array[start_row:end_row, start_column:end_column]
```

```
In [16]: #We can also get an entire row
R1 = A[0,0:2]
R1
```

```
Out[16]: array([1, 0])
```

```
In [17]: #Alternative code to get an entire row
R1 = A[0,:]
R1
```

Out[17]: array([1, 0, 0])

EXERCISE: Subset the matrix A to get this 2x2 matrix B

$$B = B_{m,n} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

```
In [18]: import numpy as np

# Define matrix A
A = np.array([[1, 2, 0], [5, 1, 2]])

# Subset matrix A to get the 2x2 matrix B
# We take the first two rows and the first two columns
B = A[:2, :2]

# Display the results
print("Matrix A:")
print(A)
print("\nMatrix B (Subset of A):")
print(B)
```

Matrix A:

```
[[1 2 0]
 [5 1 2]]
```

Matrix B (Subset of A):

```
[[1 2]
 [5 1]]
```

EXERCISE 2: Subset the matrix A to get this new matrix C:

$$C = C_{m,n} = \begin{bmatrix} A_{13} \\ A_{23} \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \end{bmatrix}$$

```
In [19]: import numpy as np

# Define matrix A
A = np.array([[1, 8, 2], [5, 1, 2]])

# EXERCISE 1: Subset matrix A to get the 2x2 matrix B
# We take the first two rows and the first two columns
B = A[:2, :2]

# EXERCISE 2: Subset the matrix A to get this new matrix C
# Matrix C consists of the elements A13 and A23 (index [0,2] and [1,2])
# We want a 2x1 matrix, so we take all rows and only the last column
# To keep it as a 2D matrix (2x1), we use a slice for the column
C = A[:, 2:3]

# Display the results
print("Matrix A:")
print(A)
print("\nMatrix B (2x2 Subset):")
print(B)
print("\nMatrix C (2x1 Subset):")
print(C)
```

Matrix A:

```
[[1 8 2]
 [5 1 2]]
```

Matrix B (2x2 Subset):

```
[[1 8]
 [5 1]]
```

Matrix C (2x1 Subset):

```
[[2]
 [2]]
```

Reducing Numpy Arrays (Reduction)

Let's define a matrix A, where we want to get the sum of the all of the values in each column. That is:

$$A = A_{m,n} = \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \end{bmatrix}$$

$$S = S_n = [A_{11} + A_{21} \quad A_{12} + A_{22} \quad A_{13} + A_{23}]$$

Reduction operations act along one or more axes. In this example, an array is summed along the axis 0 to produce a vector.

```
In [20]: A = np.array([[1,0,0],[0,1,2]])
          S = sum(A)
          print(S)
```

```
[1 1 2]
```

Broadcasting

Let's explore broadcasting in the element-wise multiplication of two-dimensional arrays:

$$B \circ C$$

In code: `B * C`

$$B \circ C = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \circ \begin{bmatrix} 0 \\ 2 \end{bmatrix}$$

```
In [21]: B*C
```

```
Out[21]: array([[ 2, 16],
                [10,  2]])
```

Matrix Multiplication

Let's explore a matrix multiplication

$$B \cdot C$$

in code `B @ C`.

$$B \cdot C = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 2 \end{bmatrix}$$

```
In [22]: B@C
```

```
Out[22]: array([[18],  
               [12]])
```

Question: What is the difference between broadcasting and matrix multiplication?

Rta: La multiplicación matricial es una operación del álgebra lineal que combina filas y columnas mediante sumas de productos y requiere dimensiones compatibles. El broadcasting es una regla computacional que permite realizar operaciones elemento a elemento entre arreglos de distinto tamaño ajustando dimensiones automáticamente. La primera tiene significado matemático formal, mientras que la segunda es solo una optimización para facilitar cálculos numéricos.